

sigstore-rust

Security Assessment

October 15, 2026

Prepared for:
Wolf Vollprecht
Sigstore

Organized by the Open Source Security Foundation

Prepared by: **Scott Arciszewski and Andrew Pan**

Table of Contents

Table of Contents **1**
Project Summary **2**
Executive Summary **3**
Project Goals **5**
Project Targets **6**
Project Coverage **7**
Summary of Findings **8**
Detailed Findings **10**
1. Duplicate getters for transparency key information 10
2. Verification is unsafe with skip_tlog 12
3. Verifier uses Rekor integratedTime before SET verification 13
4. Verifier only checks certificate against earliest timestamp in bundle 15
5. Trust root validity period checks are incorrect 17
6. Inconsistent API surface for signature verification 19
7. Default verifier configuration accepts future Rekor integrated time 21
8. Raw artifact signing performs potentially blocking actions in async context 23
9. DSSE timestamps can authenticate a different signature than payload verification 25
10. Offline TUF mode trusts mutable cached targets without authentication 27
11. TSA timestamps can borrow an unrelated authority's validity interval 29
12. Verifier expects proposed in-toto entries instead of canonical log entries 32
13. Delegation wildcards can match path separators 34
14. Cached delegated roles can bypass a delegator's authority 35
A. Vulnerability Categories **37**
B. Code Quality Recommendations **39**
About Trail of Bits **42**
Notices and Remarks **43**

Project Summary

Contact Information

The following project manager was associated with this project:

Amanda Stickler, Project Manager
amanda.stickler@trailofbits.com

The following consultants were associated with this project:

Andrew Pan, Consultant
andrew.pan@trailofbits.com

Scott Arciszewski, Consultant
scott.arciszewski@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
July 6, 2026	Pre-project kickoff call
July 10, 2026	Delivery of status update #1
July 27, 2026	Status update meeting
July 28, 2026	Delivery of status update #2
August 3, 2026	Delivery of report draft; report readout meeting
October 15, 2026	Delivery of comprehensive report

Executive Summary

Engagement Overview

The Sigstore Project engaged Trail of Bits to review the security of the sigstore-rust SDK, a library that creates and verifies Sigstore digital signatures. Sigstore signatures are used to assert facts about software provenance.

A team of two consultants conducted the review from July 6 to August 3, 2026, for a total of 3.5 engineer-weeks of effort. Our testing efforts focused on the cryptographic correctness of the signature verification flow and adherence to published specifications more broadly. With full access to source code and documentation, we performed static and dynamic testing of the sigstore-rust SDK, using automated and manual processes.

Observations and Impact

We found that sigstore-rust has been adequately tested and passes the official Sigstore conformance test suite. As such, common verification flows behave correctly, but non-PGI configurations are problematic: TOB-SIGSTORE-2 allows users to skip transparency log verification in support of a specific private deployment; and TOB-SIGSTORE-11 requires multiple trusted TSAs.

More broadly, a Sigstore signature is an attestation bound to a specific identity and time. We did not identify any misbindings over identity data, but we identified a plurality of issues surrounding signing time establishment. With these issues, an adversary may be able to present expired signing credentials as valid, but such attacks ultimately require the leakage of said signing credentials.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that the sigstore-rust maintainers take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Expand the Sigstore conformance suite.** Generate test materials from findings where possible to validate fixes.
- **Reuse and abstract where possible.** TOB-SIGSTORE-1, TOB-SIGSTORE-5, and other findings relate a tendency to copy or rewrite code instead of abstracting. To keep critical verifier codepaths clear and auditable, take measures to reuse existing code when adding new functionality.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	4
Low	2
Informational	8
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	2
Configuration	2
Cryptography	6
Data Validation	2
Denial of Service	1
Timing	1

Project Goals

The engagement was scoped to provide a security assessment of the sigstore-rust SDK. Specifically, we sought to answer the following non-exhaustive list of questions:

- Does the verification flow correctly harness cryptographic primitives?
- Can an adversary craft a bundle that improperly validates against a given identity?
- Do the signing and verification flows securely establish a root of trust?
- Is the API surface resilient to misconfigurations that reduce user security?
- Does the verification flow correctly interpret signing time evidence?
- Does the verification flow cryptographically validate inputs before interpreting them where possible?
- Are there any memory safety issues present in the codebase?
- Are the continuous integration and release workflows hardened against supply chain attacks?
- Does the codebase conform to Sigstore best practices?

Project Targets

The engagement involved reviewing and testing the following target.

sigstore-rust

Repository	https://github.com/sigstore/sigstore-rust
Version	`6d268a5d8005e5c3e91a44c20fcfd557f514e668`
Type	Rust

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- A review of the trust root establishment process
- A review of transparency log entry verification
- Dynamic testing of transparency log entry verification
- A review of the cryptographic helpers used by the verification flow
- A review of serialization and deserialization in the verification flow
- A review of user-configurable options in the verification flow
- A review of `async` API surfaces
- A cryptographic review of DSSE signature verification
- A cryptographic review of the Merkle tree proof implementation
- A cryptographic review of RFC 3161 timestamp verification

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- `sigstore-sign`
- `sigstore-cache`

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Duplicate getters for transparency key information	Configuration	Informational
2	Verification is unsafe with skip_tlog	Cryptography	Low
3	Verifier uses Rekor integratedTime before SET verification	Timing	Informational
4	Verifier only checks certificate against earliest timestamp in bundle	Cryptography	Low
5	Trust root validity period checks are incorrect	Cryptography	Informational
6	Inconsistent API surface for signature verification	Data Validation	Informational
7	Default verifier configuration accepts future Rekor integrated time	Configuration	Informational
8	Raw artifact signing performs potentially blocking actions in async context	Denial of Service	Informational
9	DSSE timestamps can authenticate a different signature than payload verification	Cryptography	Medium
10	Offline TUF mode trusts mutable cached targets without authentication	Cryptography	Medium
11	TSA timestamps can borrow an unrelated authority's validity interval	Cryptography	Medium
12	Verifier expects proposed in-toto entries instead of canonical log entries	Data Validation	Informational

13	Delegation wildcards can match path separators	Access Controls	**Informational**
14	Cached delegated roles can bypass a delegator's authority	Access Controls	**Medium**

Detailed Findings

1. Duplicate getters for transparency key information

Severity: **Informational**	Difficulty: **Undetermined**
Type: Configuration	Finding ID: TOB-SIGSTORE-1
Target: `crates/sigstore-trust-root/src/trusted_root.rs`	

Description

The `TrustedRoot` type contains many duplicate getters for Rekor and Fulcio transparency log keys. These functions map public key IDs (SHA-256 hashes of key contents) to their corresponding keys with minor differences: some functions return key materials matching the given criterion, while others return mappings represented as `HashMap` or `Vec<(...)>`. The keys themselves are either `Vec<u8>` or `DerPublicKey` (both SPKI-format) while key IDs are represented as `String`, `KeyHint`, or `LogKeyID`:

```
pub fn rekor_keys(&self) -> Result<HashMap<String, Vec<u8>>> {
    // ...
}

pub fn rekor_keys_with_hints(&self) -> Result<Vec<(KeyHint, DerPublicKey)>> {
    // ...
}

pub fn rekor_key_for_log(&self, log_id: &LogKeyId) -> Result<DerPublicKey> {
    // ...
}

pub fn rekor_key_for_log_at(&self, log_id: &LogKeyId, time: Timestamp) ->
Result<DerPublicKey> {
    // ...
}

pub fn ctfe_keys(&self) -> Result<HashMap<LogKeyId, DerPublicKey>> {
    // ...
}

pub fn ctfe_keys_with_ids(&self) -> Result<Vec<(Vec<u8>, DerPublicKey)>> {
    // ...
}
```

Figure 1.1: Interfaces for retrieving transparency key material in `sigstore-trust-root` (`sigstore-rust/crates/sigstore-trust-root/src/trusted_root.rs#L270–L395`)

While this issue is currently not exploitable, the number of competing methods available makes it difficult to determine which should be used for a given type of transparency key.

Recommendations

Short term, collapse the duplicate getters into one getter per transparency key type: `ctfe_keys` and `rekor_keys`. Consider using the existing unused `Keyring` type in `sigstore-crypto` to represent mappings:

```
fn ctfe_keys(&self) -> Result<Keyring> {
    Keyring::from_keys(self.ctlogs.iter().map(|tlog| tlog.public_key))
}
```

Figure 1.2: Key parsing logic can be delegated to a Keyring constructor

In this case, `Keyring::from_keys` should convert raw DER bytes to `ParsedPublicKey`, storing the parsed keys alongside their IDs and validity periods. `Keyring` should implement a single function for verifying transparency log entries:

```
fn verify(
    &self,
    key_id: LogKeyId,
    data: &[u8],
    signature: &SignatureBytes,
    // entry's integration time, if present
    timestamp: Option<i64>,
) -> Result<()> {
    todo!()
}
```

Figure 1.3: A proposed signature for `Keyring::verify`

`Keyring::verify` should start by looking up a candidate key by the input ID. The key's validity window should be checked against the current time and the input timestamp: the key should not have a validity period in the future, and the integration time should lie within the validity window.

Long term, when adding new code, consider the code that already exists and whether it might be reused. Also, regularly review code and look for opportunities for consolidation. Doing so will reduce code sprawl and help to keep code manageable.

2. Verification is unsafe with skip_tlog

Severity: **Low**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-SIGSTORE-2
Target: `crates/sigstore-verify/src/verify.rs`	

Description

`VerificationPolicy::skip_tlog` configures the verifier to skip Rekor transparency log entry verification via the `verify_tlog` flag:

```
if policy.verify_tlog {
    let integrated_time = crate::verify_impl::tlog::verify_tlog_entries(
        bundle,
        &self.trusted_root,
        cert_info.not_before,
        cert_info.not_after,
        policy.clock_skew_seconds,
    )?;
    // ...
}
```

Figure 2.1: Transparency log verification is gated behind the `verify_tlog` flag (sigstore-rust/crates/sigstore-verify/src/verify.rs#L333–L345)

With this configuration, the client will accept bundles without verifying Rekor entries to ensure that the signature event was logged. Additionally, a transparency log's `integratedTime` is used even when verification of transparency log entries is disabled.

Exploit Scenario

An attacker modifies a bundle with a Rekor v1 entry, backdating the signature time by modifying `integratedTime`. A downstream application that defines a policy with `skip_tlog` will treat the tampered Rekor materials as trusted and verify the bundle with the backdated validation time.

Recommendations

Short term, remove `skip_tlog`. If it is necessary for the proper functioning of integration tests, restrict its visibility to external users and document that it is unsafe.

Long term, review integration tests to ensure that they reflect real-world configuration as closely as possible.

3. Verifier uses Rekor integratedTime before SET verification

Severity: **Informational**	Difficulty: **Undetermined**
Type: Timing	Finding ID: TOB-SIGSTORE-3
Target: crates/`sigstore-verify/src/verify.rs`	

Description

The verifier calls `determine_validation_time` before transparency log verification and uses the result as a trusted time in certificate validation:

```
let validation_time = crate::verify_impl::helpers::determine_validation_time(
    bundle,
    &signature,
    &self.trusted_root,
)?;

// (1): Verify that the signing certificate chains to the root of trust,
//      is valid at the time of signing, and has CODE_SIGNING EKU.
if policy.verify_certificate {
    crate::verify_impl::helpers::verify_certificate_chain(
        &bundle.verification_material.content,
        validation_time,
        &self.trusted_root,
    )?;

    // Also verify the certificate is within its validity period
    crate::verify_impl::helpers::validate_certificate_time(validation_time,
&cert_info)?;
}
```

Figure 3.1: Verification of certificates against validation time (`sigstore-rust/crates/sigstore-verify/src/verify.rs#L263–L280`)

`determine_validation_time` verifies the structure of Rekor entries through a helper, but returns an `integratedTime` without checking the digital signature that authenticates it:

```
// Only V1 entries (0.0.1) with inclusion promises are valid timestamp sources
let is_v1 = entry.kind_version.version == "0.0.1"
    && (entry.kind_version.kind == "hashedrekord" || entry.kind_version.kind ==
"dsse");

if !is_v1 || entry.inclusion_promise.is_none() {
    continue;
```

```
}

let time = entry.integrated_time;
```

Figure 3.2: The Rekor entry checks missing SET validation (sigstore-rust/crates/sigstore-verify/src/verify_impl/helpers.rs#L167–L175)

Later in the verifier, the timestamp's signature (SET) is checked through `verify_tlog_entries`:

```
if policy.verify_tlog {
    let integrated_time = crate::verify_impl::tlog::verify_tlog_entries(
        bundle,
        &self.trusted_root,
        cert_info.not_before,
        cert_info.not_after,
        policy.clock_skew_seconds,
    )?;

    if let Some(time) = integrated_time {
        result.integrated_time = Some(time);
    }
}
```

Figure 3.3: SET is verified through `verify_tlog_entries` (sigstore-rust/crates/sigstore-verify/src/verify.rs#L333–L345)

The default policy enforces `verify_tlog`, so the SET is currently checked. Non-default configurations such as TOB-SIGSTORE-2 or a future refactor may break the complex codepath between usage and validation.

Recommendations

Short term, modify the verifier to perform SET validation in `determine_validation_time` and return only verified timestamps.

Long term, review data inputs for the verifier critical path and ensure that all data is authenticated before use.

4. Verifier only checks certificate against earliest timestamp in bundle

Severity: **Low**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-SIGSTORE-4
Target: `crates/sigstore-verify/src/verify_impl/helpers.rs`	

Description

The verifier uses `determine_validation_time` to collect trusted timestamps and establish a validation time. It prioritizes a TSA timestamp if present, falling back to a timestamp from a Rekor v1 entry if the timestamp is not present:

```
pub fn determine_validation_time(
    bundle: &Bundle,
    signature: &SignatureBytes,
    trusted_root: &TrustedRoot,
) -> Result<i64> {
    // Try TSA timestamp first (most authoritative)
    if let Some(tsa_time) = extract_tsa_timestamp(bundle, signature.as_bytes(),
trusted_root)? {
        return Ok(tsa_time);
    }

    // Try integrated time from V1 tlog entries with inclusion promises
    // Per sigstore-python: integrated_time only counts if accompanied by
inclusion_promise
    if let Some(integrated_time) = extract_v1_integrated_time_with_promise(bundle) {
        return Ok(integrated_time);
    }
    // ...
}
```

Figure 4.1: Fallback logic in `determine_validation_time` (sigstore-rust/crates/sigstore-verify/src/verify_impl/helpers.rs#L201–L234)

Only the earliest timestamp from the bundle's TSA timestamps (or Rekor timestamps, if there are no TSA timestamps) is returned.

Exploit Scenario

In a Sigstore deployment that mandates multiple timestamping authorities, an attacker gains access to expired key materials and compromises a single trusted TSA. The attacker generates a bundle that includes multiple timestamps, one of which is a backdated timestamp from the compromised TSA. Upon validation of the bundle with

`sigstore-rust`, the backdated timestamp is picked as it is the earliest. Thus, the bundle is accepted despite being signed with expired key materials.

We consider this issue low severity because the Sigstore public-good deployment currently uses one timestamping authority. Private deployments or future versions of the public-good instance with additional timestamping authorities will exercise this incorrect behavior.

Recommendations

Short term, modify the verifier to check key material validity against every timestamp included in the bundle. Concretely, `determine_validation_time` should return all timestamps after validation; then, the verifier should verify the signing certificate and the entire certificate chain against each timestamp.

5. Trust root validity period checks are incorrect

Severity: **Informational**	Difficulty: **Undetermined**
Type: Cryptography	Finding ID: TOB-SIGSTORE-5
Target: `crates/sigstore-trust-root/src/trusted_root.rs`	

Description

The Sigstore trust root comprises resources that are bound to time ranges; for example, transparency logs are periodically rotated, and clients should only submit to or validate against transparency logs that are valid for a given window. This is represented by a single type in the trust root message format, which documents expected behavior:

```
// The time range is closed and includes both the start and end times,
// (i.e., [start, end]).
// End is optional to be able to capture a period that has started but
// has no known end.
message TimeRange {
        google.protobuf.Timestamp start = 1;
        optional google.protobuf.Timestamp end = 2;
}
```

Figure 5.1: Definition of `TimeRange` as used by the Sigstore trust root (protobuf-specs/protos/sigstore_common.proto#L254–L261)

`sigstore-rust` splits this validity period concept into two types: `ValidityPeriod` and `ServiceValidityPeriod`, neither of which matches `TimeRange` semantics.

`ValidityPeriod` allows for missing start times:

```
/// Whether `time` falls within this validity window.
///
/// A missing `start` or `end` bound is treated as unbounded on that side.
/// Returns an error if a timestamp is present but malformed.
pub fn contains(&self, time: Timestamp) -> Result<bool> {
    let after_start = self.start()?.map_or(true, |s| time >= s);
    let before_end = self.end()?.map_or(true, |e| time <= e);
    Ok(after_start && before_end)
}
```

Figure 5.2: Validity window helper for `ValidityPeriod` (sigstore-rust/crates/sigstore-trust-root/src/trusted_root.rs#L199–L207)

`ServiceValidityPeriod` is exclusive of its end time:

```
pub fn is_valid(&self) -> bool {
    let now = Timestamp::now();
    if now < self.start {
        return false;
    }
    if let Some(end) = self.end {
        if now >= end {
            return false;
        }
    }
    true
}
```

Figure 5.3: Validity window helper for `ServiceValidityPeriod` (`sigstore-rust/crates/sigstore-trust-root/src/signing_config.rs#L74–L85`)

Additionally, there is an unused `tsa_validity_for_time` helper.

Recommendations

Short term, implement a single function that checks time ranges according to the semantics documented in the specification. Additionally, consider using the canonical definitions for `TrustedRoot` and `SigningConfig` generated from the Protobuf specifications.

Long term, review specifications relevant to sigstore-rust. Ensure the code's behavior matches what is required by the specification.

References

- Sigstore protobuf specifications
- Sigstore architecture documentation

6. Inconsistent API surface for signature verification

Severity: **Informational**	Difficulty: **Undetermined**
Type: Data Validation	Finding ID: TOB-SIGSTORE-6
Target: `crates/sigstore-crypto/src/checkpoint.rs`, `crates/sigstore-crypto/src/verification.rs`	

Description

`sigstore-crypto` contains overlapping helpers that abstract over public key signature verification with `aws-lc-rs`.

`verify_signature_auto` is used to verify the checkpoint and SET. It accepts an unparsed DER public key and dispatches on the key type:

```
match detect_key_type(public_key) {
    KeyType::Ed25519 => verify_ed25519(public_key, signature, message),
    KeyType::EcdsaP256 => verify_ecdsa_p256(public_key, signature, message),
    KeyType::Unknown => Err(Error::Verification(
        "unsupported or unrecognized public key type".to_string(),
    )),
}
```

Figure 6.1: Implementation of `verify_signature_auto` (sigstore-rust/crates/sigstore-crypto/src/checkpoint.rs#L126–L138)

The `verify_ed25519` and `verify_ecdsa_p256` helpers trivially parse the public key to perform signature verification over the message.

`verify_signature` and `verify_signature_prehashed` are used to verify the artifact signature. They accept unparsed DER public keys and construct `VerificationKey` types to verify against:

```
VerificationKey::from_spki(public_key, scheme)?.verify(data, signature)
```

Figure 6.2: Implementation of `verify_signature` (sigstore-rust/crates/sigstore-crypto/src/verification.rs#L186–L186)

`VerificationKey::from_spki` extracts key bytes from the SPKI structure, leaving the key as unparsed bytes:

```
// ...
// Extract raw public key bytes from the BIT STRING
let raw_bytes = spki.subject_public_key.raw_bytes().to_vec();

Ok(Self {
    bytes: raw_bytes,
    scheme,
})
```

Figure 6.3: Implementation of `VerificationKey::from_spki` (sigstore-rust/crates/sigstore-crypto/src/verification.rs#L31–L37)

The verification helpers in `VerificationKey` serve the same function as `verify_signature_auto` and its associated helpers. They dispatch over key types to parse keys and verify signatures against them:

```
match self.scheme {
    SigningScheme::EcdsaP256Sha256 => {
        let key = UnparsedPublicKey::new(&ECDSA_P256_SHA256_ASN1, &self.bytes);
        key.verify(data, signature).map_err(|_| {
            Error::Verification("ECDSA P-256 SHA-256 signature invalid".to_string())
        })
    }
    SigningScheme::EcdsaP256Sha384 => // ...
    SigningScheme::EcdsaP384Sha256 => // ...
    // ...
}
```

Figure 6.4: Implementation of `VerificationKey::from_inner` (sigstore-rust/crates/sigstore-crypto/src/verification.rs#L56–L122)

Recommendations

Short term, restructure signature verification and public key handling to improve legibility. Unify signature verification behind a single helper that accepts an `aws-lc-rs` `ParsedPublicKey`. As for key parsing, the logic should be made discrete from signature verification. Ideally, public keys should be parsed as soon as they are received from external sources: TOB-SIGSTORE-1 illustrates an interface involving the `Keyring` type. This enables precise error reporting when the client is presented with malformed keys.

Long term, when handling new types of data, consider whether a more structured internal representation might be appropriate.

7. Default verifier configuration accepts future Rekor integrated time

Severity: **Informational**	Difficulty: **Undetermined**
Type: Configuration	Finding ID: TOB-SIGSTORE-7
Target: `crates/sigstore-verify/src/verify.rs`	

Description

`VerificationPolicy` provides a configurable "clock skew" parameter:

```
/// Clock skew tolerance in seconds for time validation
///
/// This allows for a tolerance when checking that integrated times
/// are not in the future. Default is 60 seconds.
pub clock_skew_seconds: i64,
```

Figure 7.1: `VerificationPolicy`'s clock skew parameter (sigstore-rust/crates/sigstore-verify/src/verify.rs#L31–L35)

In the verification flow, `check_tlog_entries` ensures that the Rekor-provided time is in the past. It also allows for entries integrated at most `clock_skew_seconds` in the future:

```
if time > now + clock_skew_seconds {
    return Err(Error::Verification(format!(
        "integrated time {} is in the future (current time: {}, tolerance: {}s)",
        time, now, clock_skew_seconds
    )));
}
```

Figure 7.2: `check_tlog_entries` checks integration time with clock skew tolerance (sigstore-rust/crates/sigstore-verify/src/verify_impl/tlog.rs#L51–L56)

Exploit Scenario

A trusted but compromised Rekor log returns a valid v1 SET whose integration time is 60 seconds in the future. The log neglects to incorporate the entry into the public view until the integration time passes. A verifier using the default policy accepts a bundle including that SET during the 60 second grace period. Third-party monitors do not see the signature event until the window elapses. This breaks the expectation that clients should only accept logged bundles.

In this scenario, the future-dated integration time must fall within the signing certificate's validity window.

Recommendations

Short term, remove the configurable clock skew grace period.

Long term, audit the verifier configuration surface. Review concrete scenarios to determine whether a parameter is appropriate for public exposure. For example, a downstream consumer that is tempted to configure `clock_skew_seconds` may be better served by correcting the underlying system clock.

8. Raw artifact signing performs potentially blocking actions in async context

Severity: **Informational**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-SIGSTORE-8
Target: `crates/sigstore-verify/src/sign.rs`	

Description

`sigstore-rust` supports two kinds of bundle contents: raw, where the signature is computed directly over the target artifact, and DSSE, where the signature is computed over an intermediate artifact that binds a hash of the target artifact. `Signer::sign` supports the former, accepting an `Artifact::Bytes` that contains the unbounded raw bytes of an artifact. Those raw bytes are synchronously hashed and signed over:

```
pub async fn sign<'a>(&self, artifact: impl Into<Artifact<'a>>) -> Result<Bundle> {
    // ...

    // 1. Generate ephemeral key pair
    let key_pair = self.generate_ephemeral_keypair()?;

    // 2. Get signing certificate from Fulcio
    let leaf_cert_der = self.request_certificate(&key_pair).await?;

    // 3. Sign the artifact
    let signature = key_pair.sign(bytes)?;

    // 4. Compute artifact hash
    let artifact_hash = sigstore_crypto::sha256(bytes);

    // ...
}
```

Figure 8.1: Synchronous operations in async `Signer::sign` (sigstore-rust/crates/sigstore-sign/src/sign.rs#L224–L274)

Additionally, the only supported signing algorithm is ECDSA P-256, which requires input entropy. As the signature is performed synchronously, the requisite random number generation is also synchronous:

```
pub fn sign(&self, data: &[u8]) -> Result<SignatureBytes> {
    let rng = SystemRandom::new();
    match self {
        KeyPair::EcdsaP256(kp) => {
            let sig = kp.sign(&rng, data)?;
```

```
            Ok(SignatureBytes::new(sig.as_ref().to_vec()))
        }
    }
}
```

Figure 8.2: Signature generates asynchronously with `SystemRandom` (`sigstore-rust/crates/sigstore-crypto/src/signing.rs#L147–L155`)

`SystemRandom` may perform I/O while generating entropy.

Exploit Scenario

A biotechnology startup hosts a Sigstore signing server. Their asynchronous request handler reads the entire input file to a buffer and passes it to `Signer::sign`. When a single user uploads a large file, other users are unable to submit files as their requests time out. Additionally, users report that signing fails on files larger than 32GB, which is the amount of memory available on the host.

Recommendations

Short term, modify the signer to dispatch the signing and hashing operations to a dedicated thread. To avoid memory exhaustion; `Signer::sign` should accept an asynchronous reader interface for the artifact instead of a buffer containing the whole artifact.

Long term, review user inputs in API surfaces usable from async contexts. Ensure that existing async functions do not perform extended computation over input without yielding. Good candidates for review include unbounded large user inputs such as `Vec<u8>`. Also review synchronous APIs that may be used asynchronously. For example, the verification flow over raw bundles may block during signature verification. `tokio-console` may be useful for runtime audits of async task timings.

References

- `tokio::task::spawn_blocking`
- tokio-console

9. DSSE timestamps can authenticate a different signature than payload verification

Severity: **Medium**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-SIGSTORE-9
Target: `crates/sigstore-verify/src/verify_impl/helpers.rs`, `crates/sigstore-verify/src/verify.rs`	

Description

The verifier can use an RFC 3161 timestamp over the first signature in a DSSE envelope to establish certificate validity while accepting a different signature as proof of payload authenticity. An attacker who gains access to a Fulcio leaf private key can therefore create a new attestation after the certificate expires and place its valid signature after a timestamped, invalid signature. This requires a multi-signature envelope and the explicit `skip_tlog` policy; a normal keyless flow should destroy the leaf key after signing.

The timestamp path always extracts the element at index 0.

```
SignatureContent::DsseEnvelope(envelope) => {
    if envelope.signatures.is_empty() {
        return Err(Error::Verification(
            "no signatures in DSSE envelope".to_string(),
        ));
    }
    Ok(envelope.signatures[0].sig.clone())
}
```

Figure 9.1: TSA input is fixed to the first DSSE signature (sigstore-rust/crates/sigstore-verify/src/verify_impl/helpers.rs#L37–L44)

The verifier later accepts any signature that verifies against the pre-authentication encoding. It does not check the identity of the timestamped element.

```
let mut any_sig_valid = false;
for sig in &envelope.signatures {
    if sigstore_crypto::verify_signature(
        &cert_info.public_key,
        &pae,
        &sig.sig,
        cert_info.key_algorithm.default_signing_scheme(),
    )
    .is_ok()
    {
```

```
        any_sig_valid = true;
        break;
    }
}
```

Figure 9.2: DSSE verification accepts any valid signature element (`sigstore-rust/crates/sigstore-verify/src/verify.rs#L356–L370`)

We reproduced acceptance via the public `Verifier`: the TSA authenticated only the invalid element zero, while element one was a valid payload signature that was created after certificate expiry. Replacing element one with invalid bytes caused rejection.

Exploit Scenario

An attacker gains access to a short-lived Fulcio leaf private key, obtains an RFC 3161 timestamp over arbitrary signature bytes during the certificate validity window, and later signs a new DSSE payload after the certificate expires. The attacker constructs a bundle with the timestamped bytes in element zero and the new valid signature in element one. The attacker submits this malicious bundle with these materials to a verifier that uses the configurable `skip_tlog` policy. The verifier uses element zero's timestamp to validate the certificate and element one to authenticate the new payload, accepting a post-expiry attestation. We emphasize that this scenario does not apply to the default sigstore-rust verification policy and requires the leakage of an ephemeral Fulcio key, thereby making exploitation difficult.

Recommendations

Short term, reject DSSE envelopes that do not contain exactly one signature whenever a single certificate and timestamp set applies to the envelope. This prevents the issue because timestamp verification and payload verification must consume the same signature entry.

Long term, refactor the code to assign a stable identity to each DSSE signature. Require certificate, timestamp, transparency log, and payload verification results to reference the same element. Add tests for reordered, duplicated, and independently timestamped DSSE signatures. This prevents users from composing security decisions across different cryptographic objects.

10. Offline TUF mode trusts mutable cached targets without authentication

Severity: **Medium**	Difficulty: **Medium**
Type: Cryptography	Finding ID: TOB-SIGSTORE-10
Target: `crates/sigstore-trust-root/src/tuf.rs`	

Description

Offline TUF mode returns target bytes from a writable cache without checking their signed target metadata, expected length, or hashes. A cache writer can therefore replace trusted material, including Rekor, Fulcio, CT, or TSA authorities, without possessing a TUF signing key. The severity is Medium only when the cache crosses a trust boundary, such as a shared CI cache, a restored cache artifact, or a supplied cache directory. Otherwise, attackers that are able to write to the cache may already have equivalent control over the application.

The online path refreshes and authenticates the TUF metadata chain before returning targets.

```
updater
    .refresh(jiff::Timestamp::now())
    .await
    .map_err(|e| Error::Tuf(format!("TUF repository load failed: {e}")))?;
Ok(updater)
```

Figure 10.1: Online loading authenticates the TUF metadata chain (sigstore-rust/crates/sigstore-trust-root/src/tuf.rs#L390–L394)

The offline path bypasses authentication and returns raw cache bytes.

```
let cached_path = cache_dir.join("targets").join(target_name);
if let Ok(bytes) = tokio::fs::read(&cached_path).await {
    return Ok(bytes);
}
```

Figure 10.2: Offline loading returns unauthenticated cache bytes (sigstore-rust/crates/sigstore-trust-root/src/tuf.rs#L412–L415)

We reproduced the issue through a standard `TrustedRoot::from_tuf` flow: we modified a cached target with an untrusted value and the public API returned it verbatim.

Exploit Scenario

An attacker poisons a shared CI cache artifact with a syntactically valid `trusted_root.json` that replaces the Rekor public key or endpoint. A later offline build restores the cache and calls `TrustedRoot::from_tuf`. The library parses and returns the attacker-controlled trust material without consulting signed target metadata, allowing downstream verification to trust an authority selected by the cache writer. Exploitation requires cache-write access at a boundary where that access does not already imply control of the verifier process.

Recommendations

Short term, make offline mode use only immutable embedded targets. This prevents a writable target cache from becoming an implicit root of trust.

Long term, add an offline workflow that authenticates cached targets and metadata with embedded roots. When designing the offline policy, consider the possibility of an outdated client time and how it might influence expiration checks. Add regression tests to ensure that the client rejects cache tampering and rollback.

11. TSA timestamps can borrow an unrelated authority's validity interval

Severity: **Medium**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-SIGSTORE-11
Target: `crates/sigstore-verify/src/verify_impl/helpers.rs`, `crates/sigstore-trust-root/src/trusted_root.rs`	

Description

The verifier checks an RFC 3161 timestamp against the validity interval of any configured timestamp authority (TSA), rather than the authority whose certificate verified the token. A token from a retained TSA signer can therefore receive temporal authorization from an unrelated TSA entry. Exploitation requires a multi-TSA trust root with differing authorization epochs and the ability to obtain or produce a valid token from a signer outside its configured epoch.

The implementation verifies the message imprint, CMS signature, and certificate chain. However, it pools the roots, intermediates, and leaf certificates from all configured TSAs before verification. The returned result includes the authenticated time, but not the identity of the signer that authenticated it. The verifier then passes only that time to a separate validity interval helper.

```
// Build verification options from trusted root
let mut opts = TsaVerifyOpts::new();

// Get TSA root certificates
if let Ok(tsa_roots) = trusted_root.tsa_root_certs() {
    opts = opts.with_roots(tsa_roots);
}

// Get TSA intermediate certificates
if let Ok(tsa_intermediates) = trusted_root.tsa_intermediate_certs() {
    opts = opts.with_intermediates(tsa_intermediates);
}

// Get ALL TSA leaf certificates (there may be multiple TSAs)
if let Ok(tsa_leaves) = trusted_root.tsa_leaf_certs() {
    opts = opts.with_tsa_certificates(tsa_leaves);
}

// Verify the timestamp response with full cryptographic validation
let result = verify_timestamp_response(ts_bytes, signature_bytes, opts).map_err(|e|
{
    Error::Verification(format!("TSA timestamp verification failed: {}", e))
```

```
})?;

// Check that the timestamp falls within the TSA's validity period from the trust
root
let within_validity = trusted_root
    .is_timestamp_within_tsa_validity(result.time)
    .map_err(|e| {
```

Figure 11.1: Timestamp verification fails to consider multiple TSAs for validity intervals (sigstore-rust/crates/sigstore-verify/src/verify_impl/helpers.rs#L78–L104)

The helper does not consider the token's identity while checking its interval. An authority with no `valid_for` restriction authorizes any timestamp for any token, and an authority with a restriction similarly authorizes a timestamp within its interval for any token.

```
/// Check if a timestamp is within any TSA's validity period from the trust root
///
/// Returns `Ok(true)` if:
/// - There are no timestamp authorities configured (no TSA verification)
/// - Any TSA has no `valid_for` field (open-ended validity)
/// - The timestamp falls within at least one TSA's `valid_for` period
///
/// Returns `Ok(false)` only if there are TSAs with validity constraints and
/// the timestamp doesn't fall within any of them.
///
/// Returns an error if a `valid_for` timestamp is present but malformed.
pub fn is_timestamp_within_tsa_validity(&self, timestamp: Timestamp) -> Result<bool>
{
    // If no TSAs are configured, no validity check needed
    if self.timestamp_authorities.is_empty() {
        return Ok(true);
    }

    for tsa in &self.timestamp_authorities {
        // If a TSA has no valid_for constraint, it's valid for all time
        let Some(valid_for) = &tsa.valid_for else {
            return Ok(true);
        };

        // Check if timestamp falls within this TSA's validity period
        if valid_for.contains(timestamp)? {
            return Ok(true);
        }
    }
```

Figure 11.2: The validity predicate accepts an interval from any configured TSA (sigstore-rust/crates/sigstore-trust-root/src/trusted_root.rs#L519–L545)

Exploit Scenario

An organization rotates a TSA certificate chain, adding a new trust root entry with a later validity epoch. The old chain is retained in the trust root to verify historical timestamps. An attacker compromises or otherwise misuses the retired TSA signer and creates a valid RFC 3161 token outside that signer's configured epoch. The verifier authenticates the token under the retired certificate, then accepts its time because the replacement TSA's interval covers it. The attacker can consequently present an unauthorized trusted signing time and may extend the effective use of signing material whose acceptance depends on that time.

Recommendations

Short term, make the verifier bind the verified timestamp result to the exact TSA trust root entry whose leaf certificate authenticated the CMS signature, and check only that entry's `valid_for` interval. Reject tokens when the verified signer cannot be mapped unambiguously to one configured authority.

Long term, restructure the timestamp verification flow to represent cryptographic identity and temporal authorization as one verification result. Add multi-authority tests covering disjoint and overlapping epochs, missing validity metadata, retired signers, unknown signers, and both validity boundaries.

12. Verifier expects proposed in-toto entries instead of canonical log entries

Severity: **Informational**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-SIGSTORE-12
Target: `crates/sigstore-rekor/src/body.rs`, `crates/sigstore-verify/src/verify_impl/rekor.rs`	

Description

Rekor v1's in-toto proposed and canonical log entry formats are intentionally different. A client submits the DSSE payload, payload type, signatures, and public keys. Rekor verifies this material and records everything **except** the full DSSE payload, returning the canonical body as such.

`sigstore-rust` treats the canonical entry as if it were the proposed entry. Its model requires the full payload and omits the payload hash, payload type, and signature public keys:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IntotoV002Content {
    pub envelope: IntotoEnvelope,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IntotoEnvelope {
    pub payload: PayloadBytes,
    pub signatures: Vec<IntotoSignature>,
}
```

Figure 12.1: Model of the in-toto proposed entry (sigstore-rust/crates/sigstore-rekor/src/body.rs#L253–L263)

The verifier consequently extracts the payload and compares it directly with the bundle, rather than computing the digest of the bundle's DSSE payload and comparing it with `payloadHash`. It also fails to compare `payloadType` or the public key stored alongside each signature:

```
let (rekor_payload_b64, rekor_signatures) = match &body {
    RekorEntryBody::IntotoV002(intoto_body) => (
        &intoto_body.spec.content.envelope.payload,
        &intoto_body.spec.content.envelope.signatures,
    ),
    // ...
```

```
};
```

Figure 12.2: Verification expects a proposed-entry payload in the canonical log body (sigstore-rust/crates/sigstore-verify/src/verify_impl/rekor.rs#L200–L210)

The Sigstore conformance suite contains a `intoto/0.0.2` fixture, `intoto-with-custom-trust-root`. However, the fixture's Rekor body contains the full DSSE payload, allowing clients to choose either interpretation.

This issue only affects Rekor v1 `intoto/0.0.2` bundles; Rekor v2 DSSE bundles use `hashedrekord/0.0.2` instead.

Recommendations

Short term, correct the model for in-toto Rekor entries. Adjust the verifier to follow this procedure to unpack and verify an entry:

- Parse `hash`, `payloadHash`, `payloadType`, and the public key in each signature.
- Verify the SHA256 hash of the decoded bundle against `payloadHash`.
- Confirm that the bundle payload matches the declared `payloadType`.
- Ensure that each signature and public key corresponds to the DSSE envelope and signing certificate.

Long term, implement a conformance test with a DSSE Rekor v1 log entry that excludes `payloadHash`.

13. Delegation wildcards can match path separators

Severity: **Informational**	Difficulty: **Medium**
Type: Access Controls	Finding ID: TOB-SIGSTORE-13
Target: `crates/sigstore-tuf/src/metadata/role.rs`	

Description

TUF defines `*` as matching zero or more characters other than `/`. This remains true when stars are adjacent, so `releases/**` should not match `releases/stable/package`.

`sigstore-tuf` passes delegation patterns to `globset`. Although `literal_separator(true)` gives an individual `*` the correct behavior, `globset` reinterprets `**` as a recursive wildcard that matches `/`:

```
let glob = globset::GlobBuilder::new(pattern)
    .literal_separator(true)
    .build()?;

if glob.compile_matcher().is_match(target_path) {
    return Ok(true);
}
```

Figure 13.1: Simplified target globbing logic

Consequently, a targets role delegated `releases/**` receives authority over every descendant of `releases/`, rather than only files in the next path segment. A compromised delegated key can publish a target in a deeper namespace controlled by another role.

We classify this issue as informational because an operator using `**` almost certainly intends to cross directory boundaries, even if the TUF specification does not define such semantics.

Recommendations

Short term, implement TUF's path-pattern grammar directly. In particular, treat every `*` independently so it can never match `/`. Add a regression test asserting that `releases/**` does not match `releases/stable/package`.

References

- TUF specification: PATHPATTERN

14. Cached delegated roles can bypass a delegator's authority

Severity: **Medium**	Difficulty: **High**
Type: Access Controls	Finding ID: TOB-SIGSTORE-14
Target: `crates/sigstore-tuf/src/client.rs`, `crates/sigstore-tuf/src/trusted.rs`	

Description

TUF authorizes delegated metadata using the keys and threshold assigned by the parent through which it is reached. The same role name may be reachable through multiple parents with different keys in a "diamond delegation" pattern. Although each target lookup starts a new delegation traversal, the client verifies a role only when its name is absent from the trusted metadata cache:

```
if role != "targets" && self.trusted.targets_role(&role).is_none() {
    // ...
    self.trusted
        .update_delegated_targets(&bytes, &role, &delegator, now)?;
}

let targets = self.trusted.targets_role(&role).expect("role just loaded");
```

Figure 14.1: A cached role bypasses verification against the current delegator (sigstore-rust/crates/sigstore-tuf/src/client.rs#L335–L352)

`TrustedMetadataSet` persists delegated metadata across searches in a map keyed only by role name:

```
pub struct TrustedMetadataSet {
    // ...
    /// Top-level targets and any delegated targets, keyed by role name.
    targets: BTreeMap<String, Metadata<Targets>>,
}
```

Figure 14.2: `TrustedMetadataSet` caches delegated metadata by role name (sigstore-rust/crates/sigstore-tuf/src/trusted.rs#L24–L31)

On a later lookup that reaches the role through a different parent, this cache bypasses that parent's key and threshold check.

Exploit Scenario

A Sigstore repository has two roles: A is authoritative for the path `trusted_root.json`, and B is authoritative for the path `example.com/keys.json`. Each delegates to a shared role `release` through their own distinct keys. The `release` role is not pinned by snapshot hash. An attacker controlling the `release` key for B serves metadata for both `example.com/keys.json` and a malicious `trusted_root.json`. Fetching B's target caches the malicious metadata, after which the same `Updater` accepts B's `trusted_root.json`, causing Sigstore to trust attacker-controlled Fulcio, Rekor, and CT keys.

In this scenario, the attacker needs to ensure that a B target is fetched before any A targets. Additionally, diamond delegations are not used in any known TUF repositories. Notwithstanding these caveats, we believe that this pattern should be explicitly handled, as it may be surprising to future TUF ecosystems.

Recommendations

Short term, adapt the TUF client to either cache verification state per delegator or reverify cached metadata against each parent before use.

Long term, investigate whether this pattern can be modeled in the TUF conformance suite.

References

- TUF specification: target roles and delegation

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

1. **TUF target paths can escape the configured target base.** In `sigstore-tuf`, `HttpRepository::fetch_target` passes target paths directly to `Url::join`:

```
fn fetch_target<'a>(&'a self, path: &'a str, max_length: u64) ->
FetchFuture<'a> {
    Box::pin(async move {
        let url = self
            .targets_base
            .join(path)
            .map_err(|e| Error::Transport(format!("invalid URL {path:?}:
{e}")))?;
        self.bounded_get(url, max_length).await
    })
   }
```

Figure B.1: `fetch_target` joins the input `path` to `targets_base` (sigstore-rust/crates/sigstore-tuf/src/client.rs#L668–L676)

Paths such as `/x` or `../x` can resolve outside the configured target directory. As the target path argument is static in `sigstore-rust`, this is purely a matter of hygiene: there are no immediate security implications unless `sigstore-rust` elects to fetch a target containing path traversals in its name. To remediate this issue, verify that the resolved URL remains under `targets_base`.

2. **The verifier does not check the DSSE payload's inner type.** DSSE bundles are expected to contain a payload in the form of an in-toto envelope, which is internally tagged with the field `payloadType`:

```
{
    "payload": "<base64 encoded body>",
    "payloadType": "application/vnd.in-toto+json",
    "signatures": [
      {
        "sig": "<base64 encoded signature>"
      }
    ]
}
```

Figure B.2: Structure of an in-toto envelope with type tag in `payloadType`

`payload` contains a Base64-encoded in-toto statement, which is itself internally tagged with the field `_type`:

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [
    {
      "name": "a.txt",
      "digest": {
        "sha256": "<sha256 digest of a.txt>"
      }
    }
  ],
  "predicateType": "<URI of predicate type>",
  "predicate": { ... }
}
```

Figure B.3: Structure of an in-toto v1 Statement, expressing a predicate in relation to a given subject

While the verifier expects `payloadType` to match the in-toto MIME type, no such check exists against the `_type` field in the statement body. The verifier unconditionally interprets it as a v1 statement. As a result, `sigstore-verify` may incorrectly interpret statement semantics if a new version is added in the future. One way to account for this is to deserialize to an enum with `serde`:

```
#[derive(Serialize, Deserialize)]
#[serde(tag = "_type")]
pub enum IntotoPayload {
    #[serde(rename = "https://in-toto.io/Statement/v1")]
    IntotoStatementV1(IntotoStatementV1),
}

#[derive(Serialize, Deserialize)]
pub struct IntotoStatementV1 {
  // subject, predicateType, predicate ...
}
```

Figure B.4: Enum with attributes for the internally tagged Statement type (`playground link`)

With this pattern, unknown variants are not representable and produce an error at the deserialization layer. This technique can also be used for the surrounding in-toto envelope.

3. **OLPC JSON canonicalization does not normalize unicode strings.** TUF metadata is authenticated by a signature over a canonicalized form. OLPC style JSON canonicalization is used instead of RFC 8785 canonicalization. The OLPC canonical JSON specification recommends that Unicode strings be normalized to Normalization Form C, as there may be many byte-level representations of a given

Unicode string. This is a code quality recommendation as the Sigstore PGI trust root is produced with `python-tuf`, which does not itself perform Unicode normalization during canonicalization.

4. **The verifier does not check EC parameters during key detection.** The `detect_key_type` method, which is used in signature verification, attempts to infer the key type from the DER `SubjectPublicKeyInfo` structure:

```
if spki.algorithm.oid == ID_ED25519 {
    KeyType::Ed25519
} else if spki.algorithm.oid == ID_EC_PUBLIC_KEY {
    KeyType::EcdsaP256
} else {
    tracing::warn!("Unknown key algorithm OID: {}", spki.algorithm.oid);
    KeyType::Unknown
}
```

Figure B.5: Excerpt of `checkpoint.rs` (`sigstore-rust/crates/sigstore-crypto/src/checkpoint.rs#L47–L54`)

Although `KeyType::EcdsaP256` is inferred directly, the `ID_EC_PUBLIC_KEY` spans many other key types. Modify the function to match on `spki.algorithm.parameters`, which encodes the exact type.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

This report is considered by Trail of Bits to be business confidential information; it is licensed to the Sigstore Project under the terms of the project statement of work and is intended solely for internal use by the Sigstore Project. Material within this report may not be reproduced or distributed in part or in whole without the express written permission of Trail of Bits.

If published, the sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.